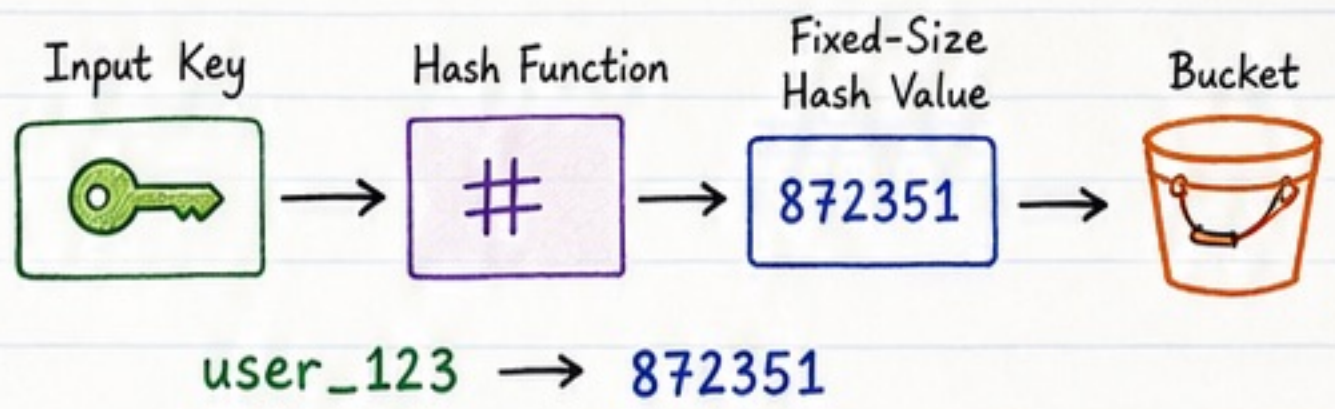


HASHING FUNDAMENTALS – QUICK NOTES ☆

1 What is Hashing?

Hashing maps input (of any size) to a fixed-size value using a hash function. It helps us quickly locate, store, and compare data.

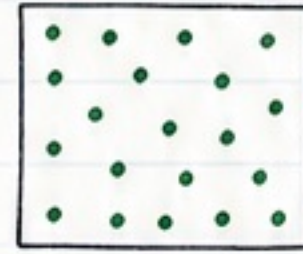


2 Good Hash Function

A good hash function should have these properties:

- ✓ Uniform
- ✓ Deterministic
- ✓ Fast
- ✓ Avalanche effect
- ✓ Fewer collisions

Good (Uniform)



Bad (Biased)



3 Hash Space & Buckets

Hash values live in a large hash space. We divide it into N buckets (indices).

$$\text{bucket} = \text{hash}(\text{key}) \% N$$



4 Collisions

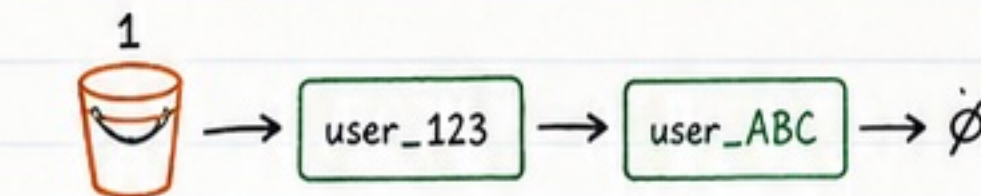
Collision: different keys, same hash.



5 Collision Handling

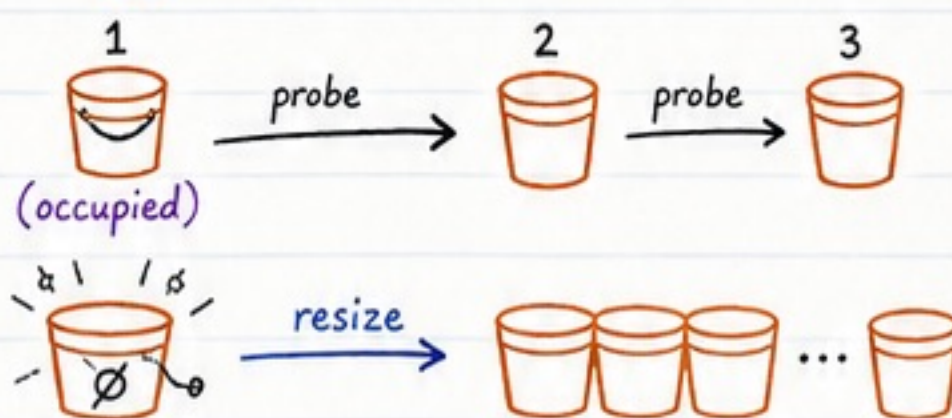
Common strategies:

① Chaining



Store multiple items in a list at the same bucket.

② Open Addressing



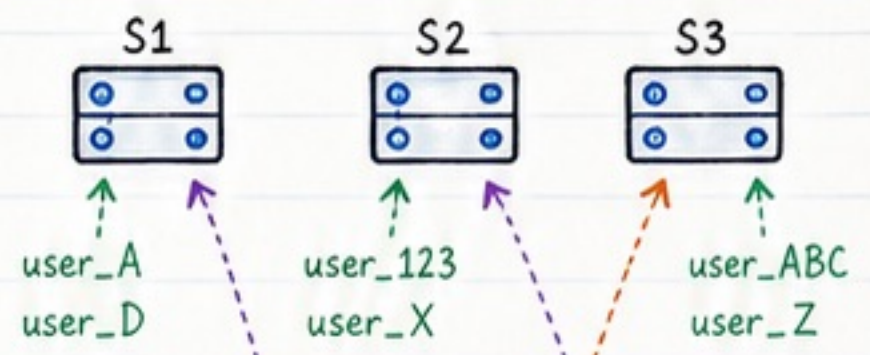
On collision, try the next available bucket (linear probing shown).

Increase N (number of buckets) and reinsert all keys.

6 Hashing in Distributed Systems

We hash a key to decide which server (shard) handles it.

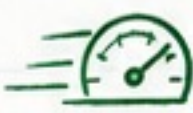
$$\text{User Key} \rightarrow \text{Hash} \rightarrow \text{Shard / Server}$$



7 General vs Cryptographic Hashing

General-purpose

- General-purpose: speed + distribution
- Used for hash tables, caches, sharding, load balancing, deduplication
- Examples: MurmurHash, FNV, xxHash



Cryptographic

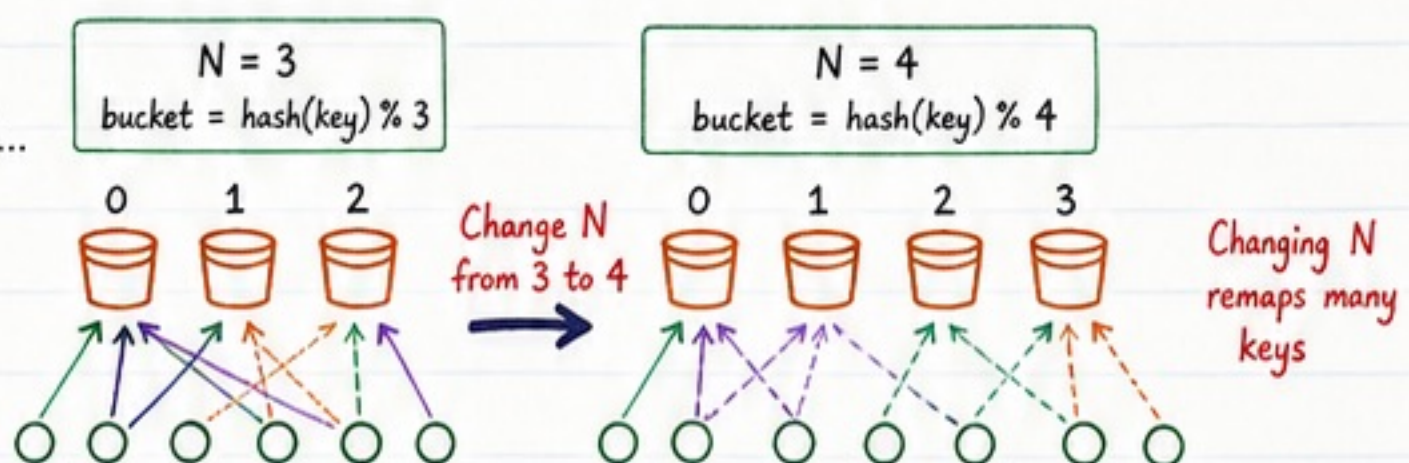
- Cryptographic: preimage + collision resistance
- Used for integrity, signatures, secure storage
- Examples: SHA-256, SHA-3



Passwords need Argon2id, scrypt, bcrypt, or PBKDF2

8 The Modulo Problem

Using $\text{bucket} = \text{hash}(\text{key}) \% N$ works well... until N changes.



Next → Consistent Hashing

GOLDEN RULE

Same key + same function = same hash.



WATCH OUT!

A hash is not encryption, and collisions are possible.



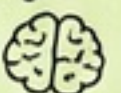
INTERVIEW TIP

Ask: Is the goal speed, routing, integrity, or password storage?



MEMORY HOOK

Hashing finds a position. Consistent hashing survives membership changes.



Day 6

